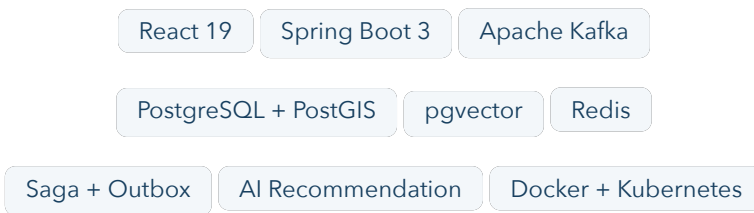


EventPulse

个性化活动发现与票务平台

详细项目规划与实施蓝图



项目定位

面向用户与活动主办方的双端平台。用户可以搜索、收藏、预订和接收余票提醒，并获得结合兴趣、距离、时间和库存的个性化推荐；主办方可以管理活动、票档、库存与实时转化数据。工程重点是 Kafka 事件流、预订 Saga、库存一致性、实时推荐反馈和可观测性。

规划假设

单人开发，基线周期 12 周，每周投入约 15-20 小时；支付使用可控的模拟网关；第一版采用模块化单体与独立推荐训练任务，避免过早拆分微服务。若时间缩短，优先保证预订闭环、Kafka 一致性和推荐评估三条主线。

Contents

1	项目目标、成功标准与边界	1
2	产品功能与用户流程	1
3	系统架构与模块边界	2
4	技术栈与版本基线	4
5	数据模型、状态机与一致性规则	5
6	API 与实时通信设计	5
7	Kafka 事件流与预订 Saga	6
8	AI 推荐系统实施方案	7
9	前端实现规划	8
10	安全、权限与数据治理	9
11	测试与质量策略	10
12	可观测性、Kubernetes 部署与 CI/CD	11
13	12 周实施路线图	13
14	验收清单与演示脚本	14
15	风险、取舍与应对	15
16	第一周可直接执行的任务清单	15
17	最终作品集产物与简历素材	15

1 项目目标、成功标准与边界

1.1 要解决的问题

通用活动平台通常依赖关键词和热门榜单，难以同时处理用户兴趣、地理距离、可参加时间、预算和实时余票。EventPulse 的目标不是简单复制票务网站，而是用一个完整业务闭环展示：**个性化发现如何影响收藏与购票，购票行为又如何通过 Kafka 反馈给推荐系统，同时在并发下保持库存正确。**

1.2 角色与核心价值

角色	核心任务	产品价值
普通用户	发现、筛选、收藏、预订、取消活动	更快找到真正可参加的活动，减少无关曝光和售罄挫败
主办方	发布活动、配置票档、管理库存、查看漏斗	准确控制库存并了解曝光到购票的转化
平台管理员	审核活动、处理异常、观察系统健康	保证内容质量、交易可追溯和服务稳定
推荐系统	召回、排序、重排并解释推荐	在相关性、距离、多样性和转化之间取得平衡

1.3 可量化成功标准

维度	MVP 验收目标	作品集展示目标
业务闭环	搜索或推荐到预订、支付模拟、出票、取消全部可演示	能展示成功、失败、超时、重复请求和库存补偿
库存正确性	任何并发场景均不超卖，已确认订单与已售库存一致	100 个并发请求下库存不为负，重复事件不重复扣减
推荐效果性能	内容基线优于热门榜单 活动查询 p95 < 250 ms；预订命令 p95 < 500 ms	NDCG@10、Recall@10 和 diversity 均有离线对照结果 给出固定硬件、数据规模、并发数和完整 benchmark 方法
事件可靠性	Outbox 无事件丢失；消费者幂等；失败进入 DLT	可展示重放、consumer lag、重试次数和 DLT 告警
质量	核心模块单元与集成测试覆盖关键分支	后端行覆盖率目标 85%+，前端关键流程覆盖率 80%+

1.4 范围与非目标

范围级别	内容
Must	认证与 RBAC、活动目录、PostGIS 地理搜索、主办方后台、票档与库存、预订 Saga、模拟支付、Kafka Outbox、幂等消费、内容推荐、行为反馈、实时余票更新、测试与监控
Should	候补队列、降价/余票提醒、推荐解释、多样性重排、主办方转化漏斗、Kubernetes 部署
Could	好友兴趣、日历同步、路线时间估计、A/B 实验平台、学习排序模型、优惠券与动态定价
明确非目标	真实银行卡收单、退款清算、座位图选座、多币种税务、完整反欺诈、与真实票务供应商双向同步

2 产品功能与用户流程

2.1 普通用户主流程

- 注册后选择兴趣类别、常驻地点、距离上限、预算和常用时间段，建立冷启动画像。
- 在首页查看“本周末附近”“与你收藏相似”“正在升温”等推荐列表；可切换地图和时间轴。
- 进入详情页查看主办方、地点、票档、剩余状态、推荐原因和时间冲突提示。
- 选择票档和数量，提交带 Idempotency-Key 的预订请求；页面实时展示预留倒计时。
- 模拟支付成功后生成电子票；失败或超时则释放库存；取消订单触发退款模拟与库存补偿。
- 曝光、点击、收藏、购票、取消和忽略事件进入 Kafka，更新实时兴趣与后续训练样本。

2.2 主办方主流程

- 创建草稿，填写标题、描述、标签、时间、场地、地理坐标、年龄与无障碍信息。
- 配置多个票档，包括价格、销售时间、总库存、单次限购和取消规则。
- 发布活动；系统异步生成 embedding、搜索索引和初始推荐候选。
- 在控制台查看曝光、点击、收藏、预订、支付成功、取消和售罄漏斗。
- 必要时暂停销售、调整可售库存或取消活动；每次变更记录审计日志并发出领域事件。

2.3 页面清单

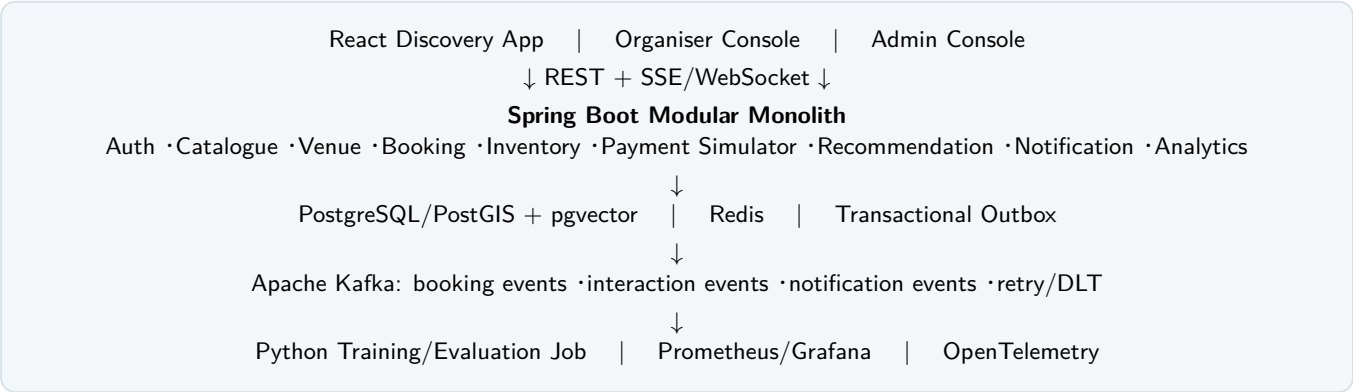
页面	主要组件	验收重点
Discovery 首页	推荐分区、筛选器、地图/列表切换、骨架屏	曝光去重、分页稳定、推荐理由可见
搜索与地图	关键词、类别、日期、价格、半径、地图标记	PostGIS 半径查询正确，URL 可分享和恢复
活动详情	图片、时间地点、票档、余票、主办方、相似活动	余票实时更新，售罄和时间冲突状态明确
结算页	订单摘要、预留倒计时、支付模拟器	刷新可恢复状态，重复提交不会重复下单
订单与票券	订单状态、二维码占位、取消入口	状态与 Saga 一致，取消规则清晰
主办方后台	活动编辑、票档库存、销售漏斗、异常列表	RBAC、乐观锁、审计记录和指标一致
管理后台	审核、用户/活动封禁、DLT 摘要、操作日志	高风险操作二次确认和完整追踪

3 系统架构与模块边界

3.1 关键架构决策

- 模块化单体优先：单个 Spring Boot 部署单元，按领域模块隔离代码和数据库访问；降低个人项目运维负担，同时保留未来拆分点。
- Kafka 承担业务异步，不承担所有读写：用户查询走同步 REST；跨模块通知、推荐反馈和预订状态推进走事件流。
- 数据库是订单与库存事实源：Kafka 传播状态变化；库存扣减仍由 PostgreSQL 原子条件更新或行锁保证。
- Transactional Outbox：业务事务与 outbox 写入同一数据库事务，再由 relay 发布 Kafka，避免数据库成功但事件丢失。
- 独立推荐训练任务：Python 负责离线训练与评估；在线服务初期由 Spring Boot 读取向量和特征，后期再增加独立推理服务。

3.2 逻辑架构



3.3 后端模块职责

模块	负责	不负责
Auth/User	注册登录、JWT、刷新令牌、RBAC、用户偏好	活动权限和订单状态
Catalogue/Venue	活动生命周期、场地、标签、搜索、发布	库存扣减和订单支付
Inventory	票档、可售量、预留、确认、释放、不变量	用户结算和支付策略
Booking	订单聚合、Saga 状态、取消规则、票券生成	直接修改活动内容
Payment Simulator	可配置成功、失败、延迟和超时	真实支付、退款清算
Recommendation	候选召回、排序、推荐理由、反馈写入	订单事实和库存事实
Notification	站内信、邮件模拟、余票与开场提醒	决定业务状态
Analytics	漏斗聚合、推荐指标、事件质量报表	作为交易事实源

3.4 建议仓库结构

eventpulse/	
apps/web/	React 19 + TypeScript
services/backend/	Spring Boot 3 modular monolith
services/recommender/	Python training and evaluation
contracts/events/	JSON Schema or Avro schemas
infra/docker/	local Docker Compose
deploy/helm/eventpulse/	reusable application chart
deploy/argocd/	ApplicationSet and environment values
ci/jenkins/	Jenkinsfile, JCasC and agent templates
observability/	Grafana dashboards and alerts
datasets/	generators and fixed benchmark seeds
docs/adr/	architecture decision records

4 技术栈与版本基线

4.1 版本选择原则

以下基线以 **2026 年 8 月 29** 日可用的稳定版本为准，兼顾 LTS、生态兼容性与作品集复现成本。表中的 x 表示锁定该版本线内的最新补丁版本；实际仓库不得使用 latest 镜像标签。Spring 生态依赖优先由 Spring Boot BOM 统一管理，避免手工拼接不兼容版本。

4.2 前端与后端应用栈

层级	技术	版本基线	用途与约束
后端运行时	Java	21 LTS	长期支持；CI 与生产使用同一 JDK 发行版
后端框架	Spring Boot	3.5.16	模块化单体、REST、Security、JPA、Kafka、Actuator
构建工具	Maven	3.9.x	使用 Wrapper；依赖版本由 Boot BOM 与 properties 管理
前端运行时	Node.js	24.20.x LTS	本地、CI 与容器统一；通过 .nvmrc 固定
前端框架	React / React DOM	19.2.x	Discovery、主办方和管理端共用组件与状态规范
类型系统	TypeScript	5.9.x	开启 strict；升级 6.x 前单独验证 breaking changes
构建工具	Vite / React plugin	8.1.x / 6.x	开发服务器、生产构建与测试环境基础
数据请求	TanStack Query	5.x	缓存、失效、重试、乐观更新和服务端状态
路由/校验	React Router / Zod	7.x / 4.x	路由、URL 状态与 API 边界校验
地图	MapLibre GL JS	5.x	地图与活动标记；底图提供方通过配置切换

4.3 数据、消息与 AI/ML 栈

层级	技术	版本基线	用途与约束
关系数据库	PostgreSQL	18.6	订单、库存、活动和 Outbox 的事实源
空间扩展	PostGIS	3.6.4	geography、GIST 索引与半径查询
向量扩展	pgvector	0.8.6	活动 embedding、近邻召回与混合推荐
事件流	Apache Kafka	4.3.1	KRaft 模式；业务事件、retry/DLT 与反馈流
缓存	Redis	8.2.x Extended	限流、短期缓存、令牌撤销与倒计时提示
ML 运行时	Python	3.13.15	推荐训练、离线评估与批处理；使用 uv 管理
语义召回	sentence-transformers	5.x	生成活动与用户兴趣 embedding
传统 ML	scikit-learn	1.8.x	数据切分、基线、指标与特征处理
学习排序	LightGBM / XGBoost	4.6.x / 3.x	V3 LambdaMART 或排序模型；MVP 可不启用
协同过滤	implicit	0.7.x	基于收藏、点击和购票隐式反馈生成候选

4.4 测试、可观测性与交付栈

层级	技术	版本基线	用途与约束
后端测试	JUnit 5 / Mockito	Boot BOM 管理	单元、切片和权限测试；不单独覆盖 BOM 版本
集成测试	Testcontainers	2.x	启动真实 PostgreSQL/PostGIS、Kafka 与 Redis
前端测试	Vitest / RTL	4.x / 16.x	hooks、组件与 API 边界测试
端到端测试	Playwright	1.62.x	Chromium 主路径；发布前增加 Firefox 冒烟测试
ML 测试	pytest	9.x	数据契约、指标计算、训练可复现性与回归测试
负载测试	k6	1.x	热门票档并发预订、幂等与超卖验证
容器	Docker Engine / Compose	29.7.x / 2.x	本地全栈与 CI 镜像构建；镜像固定 digest
编排	Kubernetes / Helm	1.36.2 / 4.2.4	作品集部署；清单在 CI 中验证
持续集成	Jenkins LTS	2.568.2	Multibranch Pipeline、JCasC 与临时 agent
持续交付	Argo CD	3.5.2	GitOps 同步、漂移检测、环境晋级与回滚
消息运维	Strimzi Operator	1.1.x	仅自建 Kafka 时使用；生产优先托管服务
供应链安全	Trivy / Cosign	0.74.x / 3.1.x	漏洞、SBOM、镜像签名与发布证明
密钥同步	External Secrets	2.10.x	从云 Secret Manager/Vault 同步到命名空间
可观测性	Prometheus / Grafana	3.x / 12.x	指标、Dashboard、告警与演示环境
链路追踪	OpenTelemetry Java Agent	2.x	REST、数据库与 Kafka 端到端 trace

4.5 锁版本与升级策略

- 前端提交 lockfile，并在 package.json 固定 package manager；后端提交 Maven Wrapper，Spring 相关依赖跟随 Boot BOM；Python 提交 pyproject.toml 与 uv.lock。
- Dockerfile 使用明确版本并在部署清单固定 image digest；开发数据库、Kafka 与 Redis 镜像和 CI 镜像保持同一版本线。
- Dependabot 或 Renovate 每周开升级 PR：补丁版本通过完整 CI 后合并，次版本需回归测试，主版本必须新增 ADR 并更新兼容性矩阵。

- 每个里程碑记录一次可复现基线：JDK、Node、Python、数据库扩展、Kafka 协议版本和模型文件 checksum 均写入构建元数据。

5 数据模型、状态机与一致性规则

5.1 核心实体

实体	关键字段	约束/索引
users	id, role, location, preference profile, status	email 唯一；软删除；偏好版本号
organisers	id, owner user, display name, verification status	owner + role 校验
venues	id, address, city, latitude/longitude, timezone	PostGIS geography GIST 索引
events	id, organiser, venue, title, description, category, time, status	status + start time；向量 HNSW/IVFFlat 索引
ticket_tiers	id, event, name, price, capacity, sale window, version	event + name 唯一；version 乐观锁
inventory	tier id, available, reserved, sold, version	available/reserved/sold 均非负；总和不超过 capacity
reservations	id, booking, tier, quantity, expires at, status	booking 唯一；expires at 索引
bookings	id, user, amount, status, idempotency key, version	user + idempotency key 唯一
payments	id, booking, attempt, status, simulated outcome	booking + attempt 唯一
interactions	user, event, type, timestamp, session, position	user/time 与 event/time 索引；分析分区表可选
outbox	event id, aggregate, type, payload, created, published	event id 唯一；published + created 索引
processed events	consumer, event id, processed at	consumer + event id 唯一

5.2 预订状态机

主路径： CREATED → RESERVING → RESERVED → PAYMENT_PENDING → CONFIRMED

失败路径： RESERVING → REJECTED；PAYMENT_PENDING → PAYMENT_FAILED → COMPENSATING → CANCELLED

超时路径： RESERVED → EXPIRED → COMPENSATING → CANCELLED

用户取消： CONFIRMED → CANCELLATION_PENDING → CANCELLED；是否归还库存由活动取消政策决定。

每个状态迁移必须满足三项条件：当前状态允许迁移、事件版本未过期、相同 eventId 未处理。非法迁移记录审计日志但不静默修改状态。

5.3 库存不变量与并发策略

- available >= 0、reserved >= 0、sold >= 0，且三者总和不超过票档容量。
- 预留使用单条条件更新：仅当 available >= requested 时减少 available 并增加 reserved；检查受影响行数判断成功。
- 确认支付将 reserved 转为 sold；失败或超时将 reserved 归还 available。每个 reservation 只允许完成一种终态。
- Kafka partition key 使用 bookingId 保证同一预订内顺序；库存命令使用 ticketTierId 便于同一票档有序处理。
- 数据库约束是最终防线，Redis 只用于短期缓存、限流和倒计时提示，不作为库存事实源。

6 API 与实时通信设计

6.1 主要 REST API

Endpoint	说明	关键要求
POST /api/v1/auth/register	注册并初始化用户	验证、限流、邮箱唯一
GET /api/v1/events	搜索、筛选和分页	cursor pagination; 稳定排序
GET /api/v1/events/nearby	半径内活动	lat/lng/radius; PostGIS
GET /api/v1/events/id	活动详情与票档状态	ETag/缓存; 已发布可见
GET /api/v1/recommendations	个性化推荐	场景、cursor、explanation
POST /api/v1/interactions	曝光/点击/收藏反馈	批量、去重、异步事件
POST /api/v1/bookings	创建预订	必须携带 Idempotency-Key
GET /api/v1/bookings/id	查询 Saga 当前状态	仅本人或管理员
POST /api/v1/bookings/id/pay	触发模拟支付	可配置结果和延迟
POST /api/v1/bookings/id/ cancel	取消预订或订单	状态校验、补偿事件
POST /api/v1/organiser/events	创建活动草稿	ORGANISER 角色
POST /api/v1/organiser/ events/id/publish	发布活动	完整性校验、Outbox
PATCH /api/v1/organiser/ tiers/id/inventory	调整可售库存	version、审计、下限校验
GET /api/v1/organiser/ analytics	转化漏斗	时间范围、事件口径固定

6.2 统一协议约定

- 错误响应使用 code/message/fieldErrors/traceId/timestamp; 领域冲突返回 409, 库存不足返回明确业务码。
- 列表使用 cursor pagination; cursor 包含最后一项排序键, 避免数据变化导致重复或漏项。
- 所有写请求携带 trace ID; 关键命令支持幂等键; PATCH 使用 version 或 If-Match 避免覆盖并发修改。
- SSE 用于预订状态与通知; WebSocket 用于活动详情页余票广播。若时间紧, 可统一为 SSE。

7 Kafka 事件流与预订 Saga

7.1 事件目录

Topic / Event	Producer	Partition key	Consumer / 用途
event.published.v1	Catalogue	eventId	Recommendation 建向量; Search 刷新索引; Notification 通知关注者
event.updated.v1	Catalogue	eventId	失效缓存、重建向量、刷新候选
interaction.recorded.v1	Web/API	userId	Profile updater、analytics、训练样本聚合
booking.created.v1	Booking	bookingId	Saga coordinator 发起库存预留
inventory.reserve.requested.v1	Booking	tierId	Inventory 原子预留
inventory.reserved.v1	Inventory	bookingId	Booking 进入 PAYMENT_PENDING; 前端更新倒计时
inventory.rejected.v1	Inventory	bookingId	Booking 标记 REJECTED 并通知用户
payment.requested.v1	Booking	bookingId	Payment Simulator 生成成功、失败或延迟
payment.completed.v1	Payment	bookingId	Booking 确认; Inventory reserved 转 sold; 出票
payment.failed.v1	Payment	bookingId	Booking 触发释放库存补偿
reservation.expired.v1	Scheduler	bookingId	Booking 与 Inventory 执行超时补偿
inventory.released.v1	Inventory	bookingId	Booking 完成取消; 可触发候补通知
booking.confirmed.v1	Booking	bookingId	Notification、analytics、recommendation feedback
booking.cancelled.v1	Booking	bookingId	Notification、analytics、负反馈特征

Topic / Event	Producer	Partition key	Consumer / 用途
notification.requested.v1	Any module	userId	Notification 发送站内信/邮件模拟

7.2 统一事件信封

```
{
  ``eventId``: ``uuid``,
  ``eventType``: ``inventory.reserved``,
  ``eventVersion``: 1,
  ``occurredAt``: ``ISO-8601``,
  ``producer``: ``inventory-module``,
  ``aggregateId``: ``booking-or-tier-id``,
  ``correlationId``: ``booking-id``,
  ``causationId``: ``previous-event-id``,
  ``traceId``: ``otel-trace-id``,
  ``payload``: { ... }
}
```

7.3 可靠性实现清单

- Outbox relay 按主键或创建时间拉取未发布事件，发布成功后标记；relay 重启允许重复发布，因此消费者必须幂等。
- 消费者在本地事务中先写 processed_events 唯一键，再执行业务更新；唯一冲突表示重复消息并安全跳过。
- 瞬时错误使用指数退避 retry topic；超过阈值进入 DLT。DLT 记录原 topic、partition、offset、错误类别和 trace ID。
- schema 只做向后兼容增加；删除或改变语义时发布 v2。契约测试阻止不兼容 schema 进入主分支。
- 监控 publish delay、consumer lag、retry rate、DLT count、duplicate count 和 Saga age。

7.4 Saga 演示场景

场景	事件结果	验收断言
正常支付	reserve → pay → confirm	一个订单、一张票、sold 正确增加
库存不足	reserve requested → rejected	无 payment；库存不变；订单 REJECTED
支付失败	reserved → payment failed → release	最终 CANCELLED；available 恢复
支付超时	reserved → expired → release	倒计时结束；迟到 payment 不得确认
重复请求	相同幂等键重复 POST	返回同一 booking；只预留一次
重复事件	同一 eventId 消费两次	第二次被幂等表拦截；状态不变
乱序事件	completed 先于 reserved 到达	非法迁移被拒绝或等待；不产生错误库存
消费者宕机	处理前/事务后/提交 offset 前宕机	恢复后最终一致；允许重复传递但不重复副作用

8 AI 推荐系统实施方案

8.1 推荐目标与约束

推荐不是单一相似度排序。目标函数同时考虑兴趣相关性、距离、时间可参加性、价格匹配、新颖性、多样性、库存可用性和预估购票概率。年龄限制、已结束、已取消、售罄、明显时间冲突属于硬过滤；其余特征进入排序与重排。

8.2 分阶段算法路线

版本	实现	目的
V0 基线	热门榜单 + 类别/距离/时间过滤	建立可比较 baseline，先验证数据与指标管道
V1 内容推荐	活动文本 embedding + 用户兴趣向量；规则加权排序	解决新用户、新活动和低交互数据问题
V2 混合排序	内容分数 + 隐式反馈协同特征 + popularity + context	利用点击、收藏、购票与取消行为提高相关性
V3 学习排序	LightGBM LambdaMART 或 XGBoost ranker	用训练样本学习非线性权重并输出特征贡献
V4 多目标重排	MMR/类别配额/距离惩罚/新颖性约束	防止列表同质化和热门活动垄断

项目必须完成 V0、V1 和 V2；V3、V4 根据时间推进。不要在行为数据不足时强行训练复杂深度模型。

8.3 候选召回与排序流程

1. **候选池**：分别从 pgvector 相似活动、关注主办方、附近热门、相似用户偏好和探索池召回，每路保留来源与原始分数。
2. **硬过滤**：移除不可见、过期、售罄、超出年龄限制、超出用户最大距离和严重时间冲突的活动。
3. **特征构建**：内容相似度、类别偏好、距离、开始时间、价格差、主办方偏好、近期热度、库存比例、曝光疲劳、用户-活动交互。
4. **排序**：先使用显式加权公式；积累训练数据后替换为 Learning-to-Rank。保留版本号以便可复现。
5. **重排**：加入多样性、探索比例和重复曝光惩罚；每个列表输出 reason codes。
6. **记录**：为每次曝光记录 request ID、模型版本、候选来源、位置和特征摘要，便于离线评估和调试。

8.4 特征与反馈权重起点

特征组	示例	初始处理
内容兴趣	类别、标签、艺人、描述 embedding	cosine similarity；近期正反馈加权
地理上下文	直线距离、城市、交通时间占位	距离分段衰减；超上限硬过滤
时间上下文	周末/工作日、时段、时间冲突	可参加性布尔 + 时间偏好分数
价格上下文	票价、用户预算、免费活动偏好	预算区间内加分，过高降权
行为反馈	click 1、save 3、purchase 6、cancel -4、dismiss -2	仅作初始启发式，后续由验证集调参
质量与探索	主办方质量、新颖度、曝光次数	质量下限 + 10% 探索槽位
实时业务	余票比例、销售速度、活动状态	售罄过滤；低库存提醒但避免误导

8.5 数据集与冷启动

- 编写固定随机种子的生成器：至少 10,000 个用户、5,000 个活动、500,000 条交互，覆盖城市、类别、价格和时间分布。
- 新用户通过兴趣问卷、常驻地点、距离和预算生成初始向量；未填写问卷时回退到附近趋势 + 多样性探索。
- 新活动通过内容 embedding、类别、地点和主办方历史质量进入候选池，不等待交互数据。
- 训练集按时间切分，避免未来行为泄漏；同一 recommendation request 的曝光与后续行为通过 request ID 关联。

8.6 评估与上线门槛

指标	含义	上线门槛
Recall@10	用户正反馈活动是否进入前 10	V1/V2 高于热门基线
NDCG@10	正反馈活动是否排在更靠前位置	V2 相对 V1 提升目标 5%+
Coverage	有机会被推荐的活动占比	不因热门偏置显著下降
Intra-list diversity	同一列表类别/向量差异	高于仅按相关性排序版本
Average distance	推荐活动平均距离	不以牺牲 NDCG 为代价无限降低
Calibration	推荐类别分布与用户兴趣分布接近度	对重度用户输出对照图
Serving p95	在线推荐响应延迟	缓存命中 < 150 ms；未命中 < 500 ms

8.7 推荐解释

解释优先由可验证的 reason codes 生成，例如“距离你 2.1 km”“与你收藏的技术会议相似”“符合周六下午偏好”。LLM 仅用于把已选理由润色成自然语言，不得编造艺人、票价、库存或距离信息；若不接入 LLM，模板解释已经满足 MVP。

9 前端实现规划

9.1 技术与状态管理

- React 19 + TypeScript + Vite；TanStack Query 管理服务端状态；React Router 管理路由；Zod 校验 API 边界。
- 地图可采用 MapLibre GL + OpenStreetMap 数据；避免在 MVP 中依赖昂贵地图 API。
- 认证 token、用户偏好和 UI 状态分离；订单状态以服务端为准，前端不得自行推导最终成功。
- 查询 key 包含所有筛选条件；mutations 成功后精确失效相关缓存，不全局刷新。

9.2 关键交互细节

- 推荐卡片进入视口达到阈值后发送一次 impression；使用 request ID + event ID 去重。
- 预订按钮点击后立即禁用，并复用同一个幂等键；网络重试不能生成新订单。
- 预留倒计时来自服务端 expiresAt，本地只负责显示；恢复页面时重新查询当前状态。
- SSE/WebSocket 断线使用指数退避重连，并在重连后执行一次 REST 状态同步。
- 无障碍目标：键盘可操作、可见焦点、表单错误关联、地图外提供等价列表、颜色对比达到 WCAG AA。

10 安全、权限与数据治理

领域	控制措施	验收方式
认证	短期 access JWT + 旋转 refresh token；Redis revocation	过期、撤销、重复刷新和登出测试
授权	USER/ORGANISER/ADMIN；资源所有权检查	跨主办方修改必须 403
输入安全	Bean Validation、白名单枚举、上传限制、HTML 清洗	恶意 payload 与存储型 XSS 测试
接口滥用	IP/user 限流；登录、反馈和预订不同配额	Redis 限流集成测试
交易审计	订单、库存调整、活动状态和管理员操作留痕	trace ID 可串联完整 Saga
隐私	只保存必要位置偏好；精确位置可降精度；提供数据删除	删除后个人数据与推荐画像不可查询
Secrets	本地.env.example；CI/K8s secret；日志脱敏	仓库扫描无密钥，日志无 token

11 测试与质量策略

11.1 测试分层

层级	工具与范围	必测内容
单元测试	JUnit 5、Mockito、Vitest	排序公式、状态机、取消规则、库存计算、前端 hooks
切片测试	MockMvc、Spring Security Test、React Testing Library	Controller 验证、权限、错误响应、关键组件交互
数据库集成	Testcontainers PostgreSQL/PostGIS	条件扣减、唯一键、Flyway、地理查询、向量查询
Kafka 集成	Testcontainers Kafka	Outbox relay、幂等消费、retry/DLT、partition key
端到端	Playwright	注册、搜索、预订成功/失败、取消、主办方发布
推荐评估	pytest + 固定数据集	指标可复现、时间切分、基线对照、无数据泄漏

11.2 并发与故障测试

- 使用 k6 或 Gatling 对单一票档发起并发预订，验证无超卖、成功数等于库存、重复幂等键只成功一次。
- 在 outbox 发布前、消费者业务事务后和 offset 提交前模拟进程终止，验证恢复后的最终一致性。
- 注入 Kafka 延迟、消费者暂停、支付超时和数据库短暂不可用，观察 retry、DLT、Saga age 与告警。
- 对 5,000 活动、500,000 交互和 100 并发客户端进行固定 benchmark；报告 warm-up、运行次数和硬件环境。

11.3 Definition of Done

每个用户故事只有在以下条件全部满足时才算完成：功能与异常路径实现；单元/集成测试通过；权限和审计明确；API/事件契约更新；关键指标可观察；README 有运行说明；没有未解释的 flaky test 或 DLT 消息。

12 可观测性、Kubernetes 部署与 CI/CD

12.1 关键指标与告警

Dashboard	指标	告警示例
API	request rate、error rate、p50/p95/p99、DB pool	5 分钟错误率 > 2%
Kafka	produce rate、consumer lag、retry、DLT、publish delay	lag 连续 10 分钟增长
Booking Saga Inventory	各状态数量、成功率、补偿率、Saga age available/reserved/sold、拒绝率、不变 量错误	PAYMENT_PENDING 超 5 分钟 任一库存为负立即告警
Recommendation	latency、cache hit、coverage、模型版本、feedback rate	新版本 latency 或 coverage 异常
Business	impression、CTR、save rate、purchase rate、cancel rate	数据骤降提示事件管道故障

所有日志使用 JSON，至少包含 timestamp、level、service、module、traceId、correlationId、bookingId 和 eventId。OpenTelemetry trace 串联 REST 请求、数据库事务、Kafka publish/consume 与通知。

12.2 部署建议与环境分层

建议默认方案：本地继续使用 Docker Compose；开发和演示使用一个三节点 Kubernetes 集群，通过 eventpulse-dev、eventpulse-staging 与 eventpulse-prod 命名空间隔离环境。生产型演示中，应用运行在集群内，PostgreSQL 与 Kafka 优先使用托管服务；只有当“在 Kubernetes 上运维有状态系统”本身是展示目标时，才使用 PostgreSQL Operator 与 Strimzi 自建。

环境	目标	建议拓扑	发布规则
Local	开发与快速反馈	Compose；单节点 Kafka/PostgreSQL/Redis	手动启动；固定 seed 数据
Dev	每次 main 合并验证	共享集群独立 namespace； 最小副本	Argo CD 自动同步、自愈与清理
Staging	生产前验收与压测	与生产同 Chart；独立数据 库/schema 和 topic 前缀	自动同步；E2E、迁移与冒烟测试通过
Prod/Demo	稳定作品集环境	三个 worker 节点；应用双副 本；托管数据层优先	受保护 PR 晋级；人工批准或维护窗口

集群入口采用 **Gateway API** 配合一个持续维护的 Gateway Controller，并由 cert-manager 管理 TLS。不要新建 ingress-nginx 方案，因为该项目已在 2026 年 3 月结束维护。DNS 可由 external-dns 自动管理；云环境使用托管 LoadBalancer，本地演示使用端口映射或 MetalLB。

12.3 分布式演进边界

目标不是一次性拆成大量微服务，而是先形成清晰边界、独立镜像与事件契约，再按负载和故障隔离需求拆分。**Booking 与 Inventory 在第一阶段保留同一事务边界**，避免为了分布式而增加最危险的一致性问题。

阶段	部署单元	适用条件	核心验收
Stage 0	web、backend、recom-mender job	MVP 与前 6 周；模块化单体	模块依赖受控；Outbox/Saga 已真实运行
Stage 1	web、core-api、recommenda-tion、notification worker	推荐计算或通知负载需要独立 扩缩	每个单元独立镜像、HPA、Dash-board 和回滚
Stage 2	identity、catalogue-search、booking-inventory、recom-mendation、notification	演示分布式故障隔离与独立发 布	API/事件契约兼容；跨服务 trace 完整
Stage 3	booking 与 inventory 分离；an-alytics 独立	仅在并发与容量测试证明有必要 时	Saga 补偿、幂等、重放和灾难恢 复均自动化

建议的长期服务边界是：Identity、Catalogue/Search、Booking/Inventory、Recommendation、Notification 与 Analytics。同步调用只用于立即需要结果的查询或命令；状态传播、推荐反馈、通知与分析使用 Kafka。每个服务只能写自己的表或 schema，不得跨服务直接更新；事件包含 eventId、版本、trace ID 与 schema version。

12.4 Kubernetes 工作负载映射

组件	Kubernetes 资源	副本/扩缩	持久化与可靠性
web	Deployment + Service	2; CPU HPA 2-4	无状态; CDN 缓存静态资源
core/API 服务	Deployment + Service	2; CPU/延迟 HPA 2-6	RollingUpdate; PDB minAvailable=1
Kafka consumers	Deployment	1-6; 按 consumer lag 扩缩	同一 group; 优雅停止后提交 offset
recommender API	Deployment + Service	2; CPU/延迟 HPA	模型只读挂载或从对象存储下载
训练/评估	CronJob / Job	单次任务; 禁止并发重叠	产出模型、指标与 checksum 到对象存储
数据库迁移	PreSync Job	每版本一次	Flyway expand/contract; 失败阻止后续 wave
PostgreSQL	托管服务优先	Multi-AZ/自动备份	PITR、连接池、PostGIS/pgvector 扩展
Kafka	托管或 Strimzi	3 broker/controller 起	replication factor 3; min ISR 2; 跨节点分布
Redis	托管或 Sentinel	主从/故障转移	只保存可重建状态, 不承担库存事实

每个在线 Deployment 都必须配置 requests/limits、startup/readiness/liveness probes、PodDisruptionBudget、topology spread constraints、专用 ServiceAccount 与默认拒绝的 NetworkPolicy。HPA 使用 autoscaling/v2; API 依据 CPU 和请求延迟扩缩, Kafka consumer 更适合依据 consumer lag 扩缩。发布时设置 maxUnavailable=0、maxSurge=1, 并保留足够的 termination grace period 完成请求和消费者收尾。

12.5 集群平台组件清单

类别	必需组件	说明
流量与证书	Gateway Controller、Gateway API、cert-manager	HTTPS、HTTPRoute、证书续期与流量切分
发布与配置	Argo CD、Helm、ApplicationSet	Git 为期望状态; 环境 values 分离
密钥	External Secrets + Vault/云 Secret Manager	Git 中只保存引用; 禁止明文 Secret
可观测性	Prometheus、Grafana、Loki、Tempo/OTel Collector	metrics、logs、traces 使用同一 trace ID 关联
弹性	metrics-server、HPA、PDB、topology spread	扩缩容、维护与单节点故障保护
安全	RBAC、NetworkPolicy、Pod Security、Trivy、Cosign	最小权限、镜像扫描、签名与准入校验
备份	数据库 PITR、对象存储版本、Git 与 Jenkins 配置备份	定期执行恢复演练, 不只验证备份任务成功

12.6 Jenkins: Continuous Integration 方式

Jenkins 负责构建、测试、安全检查和发布制品, 不直接成为生产集群的部署控制器。Controller 通过官方 Helm Chart 安装到 ci namespace, 配置 PVC 与定期备份; 插件列表和全局配置通过 JCasc 版本化。Kubernetes plugin 为每次构建创建临时 agent Pod, 分别包含 Maven/JDK、Node、Python 和镜像构建工具容器, 任务结束即销毁。

使用 GitHub App/Webhook + Multibranch Pipeline 自动发现 PR 与分支; 根目录 Jenkinsfile 使用 Declarative Pipeline。公共逻辑可放受控 Shared Library, 但个人项目第一版优先保持 Jenkinsfile 可读。不要给 Jenkins cluster-admin, 也不要通过特权 Docker-in-Docker 构建镜像; 使用 rootless BuildKit 或等价 OCI builder, 并仅授予推送目标镜像仓库的权限。

Stage	执行内容	失败策略/产物
元数据	checkout、版本号、commit SHA、变更路径识别	生成统一 build metadata
Fast gates	lint、format、type-check、backend/frontend/ML unit; 并行执行	任一失败立即停止
Integration	Testcontainers DB/Kafka/Redis、契约测试、Flyway 验证	JUnit/coverage/contract reports
E2E/性能	Playwright 主路径、k6 smoke; 夜间执行完整 benchmark	截图、trace、性能趋势
Security	dependency、secret、laC、image scan; 生成 CycloneDX/SPDX SBOM	Critical 阻断; High 需有到期豁免
Build/Publish	多阶段镜像、commit SHA tag、registry push、Cosign 签名	记录不可变 digest 与 provenance
GitOps update	修改 dev values 中的 image digest 并提交/开 PR	Jenkins 不调用生产 kubectl

PR 只执行验证且不发布正式镜像; 合并 main 后发布 sha-<commit> 镜像并更新 dev; 语义化 tag 触发 release

candidate、完整 E2E 与供应链签名。Jenkins 凭据只保存 GitHub App、Registry 和 GitOps repo 的最小权限 token；云密钥通过 workload identity 或短期凭据获得。

12.7 Argo CD: GitOps Continuous Delivery

推荐将应用源码与部署期望状态分离：源码仓库包含 Dockerfile、Jenkinsfile 和 Helm Chart；独立 eventpulse-deploy 仓库保存 dev/staging/prod values、ApplicationSet 与集群级组件版本。若暂时只维护一个仓库，也应将 deploy/argocd 和 deploy/helm 与应用源码清晰隔离并设置 CODEOWNERS。

```
Pull request → Jenkins quality gates
↓
main merge → build once → scan + SBOM + sign → push immutable image digest
↓
Jenkins updates GitOps dev values → Argo CD syncs dev → smoke/E2E
↓
promotion PR copies the same digest to staging/prod → approval → Argo CD sync
```

Argo CD 使用 ApplicationSet 生成三个环境的 Application。dev 与 staging 开启 automated sync、self-heal 和受控 prune；prod 通过受保护 PR 决定版本，合并后可自动同步，也可保留人工 Sync 窗口。禁止在集群内手工修改长期配置；紧急修改后必须立即回写 Git，否则 self-heal 会恢复声明状态。

使用 sync waves 保证顺序：-2 安装 CRD/operator，-1 执行数据库迁移，0 部署 API/worker，1 部署 web/route，PostSync Job 执行冒烟测试。第一版使用 Deployment RollingUpdate；当指标稳定后，可选 Argo Rollouts 做 10% → 50% → 100% 金丝雀，并以错误率、p95 与 Saga 失败率作为晋级门槛。

回滚使用 Git revert 或将环境 values 恢复到上一 image digest，由 Argo CD 收敛；数据库采用 expand/contract 迁移，任何版本至少兼容前后一个应用版本。回滚演练必须覆盖镜像回退、迁移失败、Kafka schema 兼容与消费者积压恢复。

12.8 CI/CD 所需仓库资产

资产	最低要求
Jenkinsfile	Multibranch Declarative Pipeline；并行测试、报告归档、超时、重试和 post 清理
ci/jenkins/jcasc/	JCasC、插件锁定文件、Kubernetes agent Pod templates、备份/恢复说明
Dockerfile	非 root、多阶段构建、只复制运行时产物、HEALTHCHECK 与明确 base digest
deploy/helm/	每种 workload 的 Deployment/Service/HPA/PDB/NetworkPolicy/ServiceAccount 与 probes
eventpulse/	AppProject、ApplicationSet、sync policy、waves/hooks 和环境 values
deploy/argocd/	Gateway、cert-manager、External Secrets、observability 与 policy 配置
deploy/platform/	Trivy 配置、SBOM、Cosign policy、例外清单和密钥轮换流程
security/	发布、回滚、数据库恢复、Kafka lag/DLT、证书和 Jenkins/Argo CD 故障处理
docs/runbooks/	

完成门槛：从一次 main 合并开始，Jenkins 必须能生成带 digest、SBOM 与签名的不可变镜像；Argo CD 必须仅根据 Git 变更部署同一 digest；Dashboard 能显示发布前后错误率、延迟、consumer lag 与 Saga 成功率；执行 Git revert 后可在目标恢复时间内回到上一健康版本。

13 12 周实施路线图

13.1 工作量基线

预计总投入 180-240 小时。每周结束必须有可运行增量和清晰退出条件，避免到最后才集成 Kafka、测试或推荐评估。

周次	目标	主要交付物	退出条件
1	立项与基础设施	ADR、领域模型、OpenAPI 初稿、monorepo、Docker Compose、CI 骨架	web、backend、DB、Kafka、Redis 一键启动；基础 pipeline 绿
2	认证与活动目录	JWT/RBAC、主办方、场地、活动草稿/发布、Flyway	主办方只能修改自己的活动；发布校验完整
3	搜索与前端发现	PostGIS 附近搜索、筛选、cursor、地图/列表、详情页	固定数据集下查询正确；URL 状态可恢复
4	票档、库存与同步预订	ticket tier、inventory、booking 状态机、幂等 API	并发条件更新无超卖；重复请求返回同一 booking
5	Kafka 与 Outbox	event envelope、schema、outbox relay、processed events、retry/DLT	kill/restart 后事件最终到达；重复消息无副作用

周次	目标	主要交付物	退出条件
6	预订 Saga 与支付模拟	reserve、payment、confirm、release、expire、SSE 状态	八个 Saga 场景全部自动化通过
7	推荐 V0/V1	数据生成器、embedding、pgvector、内容召回、冷启动、离线指标	V1 在 Recall/NDCG 上优于热门基线
8	推荐 V2 与反馈闭环	interaction events、用户画像、混合排序、reason codes、缓存	点击/收藏/购票能影响后续推荐；版本可追踪
9	实时体验与主办方分析	余票广播、提醒、候补基础、转化漏斗	售罄状态及时更新；漏斗口径与事件数一致
10	安全、测试与故障注入	权限矩阵、限流、E2E、Kafka/DB 故障场景、覆盖率	核心质量门槛通过；无已知超卖或重复扣减
11	可观测性与部署	Prometheus/Grafana、OTel、Jenkins/JCasC、Helm、Argo CD/ApplicationSet、K8s	CI 产出签名镜像；GitOps 晋级与回滚可重复；Dashboard 可定位失败 Saga
12	Benchmark 与作品集交付	性能/推荐评估报告、README、架构图、演示脚本、简历 bullet	新机器按 README 启动；8 分钟演示稳定完成

13.2 8 周压缩路线

若只有 8 周，删除候补、降价提醒、V3 Learning-to-Rank、Kubernetes 深度配置和管理员后台；保留第 1-6 周核心，将推荐 V0-V2、测试/监控、README 与 benchmark 分别合并到最后两周。绝不能为了功能数量删掉库存并发测试、Outbox 幂等或推荐基线对照。

14 验收清单与演示脚本

14.1 MVP 发布门槛

- 用户和主办方角色流程完整；至少 5,000 个可搜索活动和 500,000 条可复现交互数据。
- 地理、时间、类别和价格筛选正确；推荐响应包含来源、模型版本和可验证解释。
- 预订成功、库存不足、支付失败、超时、取消、重复请求和重复事件均有自动化测试。
- Kafka Outbox、retry、DLT、幂等表、schema version 和 consumer lag 监控全部可演示。
- CI 从空环境完成构建、测试、镜像与部署；README 不依赖作者口头补充。
- benchmark 与推荐评估报告包含数据规模、硬件、参数、基线和重复运行结果。

14.2 8 分钟作品集演示

1. 45 秒：说明问题、角色和系统架构，强调推荐反馈与预订一致性闭环。
2. 75 秒：冷启动问卷、附近搜索、地图与推荐理由。
3. 90 秒：正常预订、实时倒计时、支付成功和余票更新。
4. 90 秒：并发抢最后库存，展示无超卖与被拒绝订单。
5. 75 秒：模拟支付失败/超时，展示 Saga 补偿、日志 trace 和库存恢复。
6. 60 秒：点击/收藏后推荐变化与离线 NDCG/Recall 对比。
7. 45 秒：Jenkins pipeline、Testcontainers、Grafana 与 consumer lag/DLT。
8. 30 秒：总结量化结果和下一步扩展。

15 风险、取舍与应对

风险	概率	影响	应对
范围过大	高	高	Must/Should/Could 严格分层；每周有退出条件；先完成闭环
Kafka 只成为装饰	中	高	让 Saga、Outbox、推荐反馈和通知真实依赖事件；演示故障恢复
库存并发错误	中	高	数据库条件更新 + 约束 + 幂等 + 高并发自动测试
推荐数据不足	高	中	固定合成数据 + 热门基线 + 内容推荐；复杂模型作为后续
指标看似漂亮但不可复现	中	高	固定 seed、时间切分、脚本化 benchmark、记录硬件和版本
运维占用过多时间	中	中	模块化单体；本地 Compose 优先；K8s 放到第 11 周
过早拆分微服务	中	高	先模块化单体；Booking/Inventory 保持事务边界；用指标触发拆分
集群有状态运维失败	中	高	生产型演示优先托管 PostgreSQL/Kafka；自建必须有 operator、备份和恢复演练
地图或外部 API 不稳定	中	中	MapLibre/OSM；路线时间先用距离近似；外部依赖可替换
第二页式功能堆砌	中	中	README 围绕三个证据：推荐提升、库存正确、事件可靠

16 第一周可直接执行的任务清单

1. 建立 monorepo、分支保护、Conventional Commits、Issue 模板和 12 周 milestone。
2. 写 5 个 ADR：模块化单体、PostgreSQL 作为库存事实源、Kafka Outbox、Saga 协调方式、推荐训练/在线边界。
3. 定义最小 OpenAPI：认证、活动搜索、活动详情、创建预订、查询预订。
4. 定义第一版事件 envelope 与 booking.created.v1 schema，并加入兼容性测试占位。
5. 建立 Flyway：users、organisers、venues、events、ticket tiers、inventory、bookings、outbox。
6. 创建 Docker Compose 和 healthcheck；加入固定 seed 的 50 个活动作为 smoke data。
7. 建立 Jenkins Multibranch pipeline：backend、frontend、recommender test、Docker build；预留 GitOps update stage。
8. 完成一个纵向切片：主办方创建并发布活动，用户在 React 列表中看到该活动。

第一周结束检查点：新的开发者只需要复制环境变量示例并运行一个命令，就能启动 React、Spring Boot、PostgreSQL/PostGIS、Kafka 和 Redis；CI 全绿；发布活动的最小纵向流程可操作。

17 最终作品集产物与简历素材

17.1 必须提交的仓库资产

- README：问题、架构、快速启动、演示账号、关键取舍、指标结果和已知限制。
- docs：架构图、ER 图、Saga 时序图、事件目录、ADR、API/事件契约、测试策略。
- reports：推荐离线评估、并发库存测试、API benchmark、故障注入结果。
- observability：Grafana dashboard JSON、Prometheus rules、示例 trace 与 DLT runbook。
- delivery：Jenkinsfile/JCasC、Helm Chart、Argo CD ApplicationSet、环境 values、SBOM/签名与回滚 runbook。
- scripts：seed、benchmark、训练、评估、环境重置和完整 demo setup。

17.2 建议简历 bullet 模板

最终数据必须来自真实 benchmark，再替换方括号中的占位值：

- Built a React/Spring Boot event discovery and ticketing platform with PostGIS geo-search and hybrid recommendations, improving NDCG@10 by [X%] over a popularity baseline across [N] interactions.
- Engineered a Kafka-driven booking Saga with Transactional Outbox, idempotent consumers, timeout compensation, and atomic inventory reservations, preventing overselling under [N] concurrent requests.
- Delivered live availability and booking updates via SSE/WebSocket, with p95 event search latency of [X ms] and end-to-end observability using Prometheus, Grafana, and OpenTelemetry.
- Automated JUnit/Mockito/MockMvc/Testcontainers, pytest, Vitest, and Playwright quality gates through Jenkins and containerised deployment on Kubernetes.

17.3 最终决策原则

如果时间或复杂度超出预期，优先级始终是：**库存正确性与 Saga 可恢复性** > **推荐系统可评估性** > **完整用户体验** > **额外功能数量**。项目价值取决于能否用测试、指标和故障演示证明业务流程与推荐闭环可靠。